

What is the SlaD-Tool?

The **S**tability and **I**nteraction **a**ssessment in the frequency-**D**omain tool (**SlaD**) is a open-source tool designed to analyze small-signal stability and interactions in large modern power systems. It helps identify potential interactions and assess closed-loop stability using advanced frequency-domain analysis techniques.

SlaD consists of two main modules:

1. Scanning Module: Determines the system's frequency response through impedance and admittance matrices.
2. Analysis Module: Applies the Generalized Nyquist Criterion and other specialized algorithms to evaluate system stability and interactions.

Key features and capabilities:

- Identification of interaction frequencies and stability across a wide frequency range (including subsynchronous and harmonic regimes).
- Determination of stability limits between interconnected systems.
- Prediction of oscillations and their damping levels.
- Analysis of multiple operating points.
- Open-source code and interface for Transmission System Operators (TSOs) and academic institutions.

Where to Get the SlaD-Tool?

The **SlaD**-Tool is freely available for download from the following GitHub repository:

<https://github.com/luisangelgare/SlaD-Tool>

As this is an open-source code, any use, modification, or contribution based on this tool must properly acknowledge the original authors and contributors. To cite this work, please use the following reference:

García-Reyes, L. A., Gomis-Bellmunt, O., Prieto-Araujo, E., Lacerda, V. A., & Chea-Mañé, M., "SIaD Tool: A Comprehensive Frequency-Domain Tool for Small-Signal Stability and Interaction Assessment in Modern Power Systems". arXiv preprint arXiv:2601.05519 (2026).

Requirements to Start Using the SlaD-Tool

PSCAD™ and Python Version:

- PSCAD™ Version 5.0.2 Professional Edition
- Python 3.11.14

Python Libraries:

- mhi.pscad 3.1.0
- numpy 2.4.1
- pandas 2.3.3
- matplotlib 3.10.8
- pathlib 1.0.1
- decimal (provided by python 3.11.14)
- os (provided by python 3.11.14)
- time (provided by python 3.11.14)

Recommended Software & Hardware:

- Spyder IDE 6
- Intel Core i5 processor or higher
- 8 GB RAM or more

How to Use the SlaD-Tool?

Once the SlaD-Tool source files have been downloaded and extracted, the usage process is divided into two separate guides depending on the working environment:

Using PSCAD™ and Python Version:

After extracting the contents of the .zip file, you will find four main files, as shown in Figure 1.

Nombre
 exe_SlaD.py
 FDScanning.py
 GNC_Module.py
 MA_Module.py
 PA_Module.py
 PM_Module.py
 SlaD_Tool.pscx
 Utilities.py

Figure 1. Files included in the **SlaD**-Tool for PSCAD™ and Python version.

Each file serves the following purpose:

- *exe_SlaD.py*: The main executable script that launches the **SlaD**-Tool and contains all necessary settings and parameters for scanning and stability analysis.
- *FDScanning.py*: The master script containing the full algorithm for open-loop system scanning. This file must be placed in the working directory without modification.
- *GNCModule.py*: Executes the Generalized Nyquist Criterion (GNC) with all configuration options preloaded. This file must remain unmodified in the working directory.
- *MAModule.py*: Modal analysis module used to perform eigenvalue-based and participation factor-based small-signal analysis. This file must remain unmodified in the working directory.
- *PAModule.py*: Passivity Assessment module for evaluating both systems. This file must remain unmodified in the working directory.
- *PMMModule.py*: Phase Margin Analysis module. This file must remain unmodified in the working directory.
- *Utilities.py*: Utilities and other functions/resources necessary for the different modules and main script. This file must be placed in the working directory without modification.
- *SlaD_Tool.pscx*: A PSCAD™ project file that includes the tool block.

Steps to Install and Use the SlaD-Tool (PSCAD™ & Python Version)

Step 1: Set Up the Working Environment

Copy the files: *exe_SlaD.py*, *FDScanning.py*, *GNCModule.py*, *MAModule.py*, *PAModule.py*, *PMModule.py* and *Utilities.py* files into your main working directory (see Figure 2). This folder should include all the necessary files to run the simulation, such as the workspace and project files.

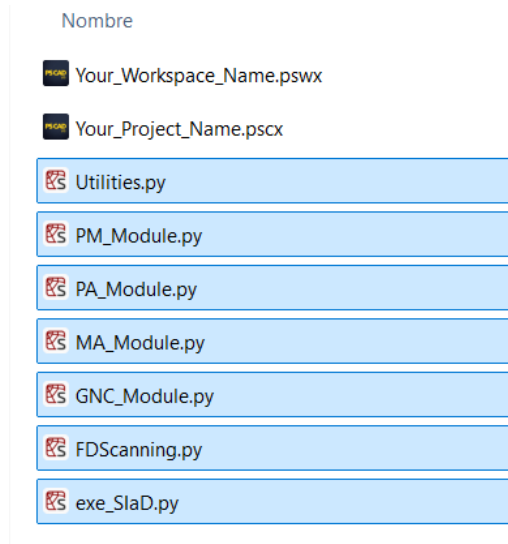


Figure 2. Copying the master files to your working directory.

Step 2: Define Global Variables in the Execution Script

Open the *exe_SlaD.py* file. Inside the script, you will find a section for defining the global variables used by the tool. **Do not modify the section IMPORTS.**

3.1 Simulation Parameters

The first set of parameters in the *exe_SlaD.py* script is divided into three sections. The variables to be defined are shown in Figure 3 and are explained below.

```
# =====
# USER-DEFINED PARAMETERS
# =====

workspace_name = 'Your_Workspace_Name' # PSCAD workspace name
project_name = "Your_Project_Name"     # PSCAD project name
fortran_ext = ".gf46"                  # Fortran or intel version: ".gf46", ".gf81", etc.
out_filename = "measurements"          # Proposed name of the file .out (must be assigned here)
results_folder = "outcomes"            # Proposed name of the results folder

Tinit = 10                             # Initialization time (s)
fs = 1                                 # Resolution frequency in Hz
f_max = 500                            # Upper limit of the frequency band
f_points = 200                         # Number of frequency points
f_scale = 1                            # Frequency vector scale type: 1 for log and 2 for linear
step_time = 50                         # Micro-seconds fixed step time for PSCAD
```

Figure 3. First block of variables definition in *exe_SlaD.py*.

The first section includes the simulation parameters, which must be carefully defined to ensure proper execution of the analysis.

- `workspace_name`: This is the name of the .pswx file containing the entire EMT model of the system to be analyzed (without the extension .pswx). For example, in the application shown in Figure 2, the file name would be "Your_Project_Name.pswx".
- `project_name`: Select and write the project name where you are applying the tool. For the moment, **SlaD-Tool** is just compatible with one additional project.
- `fortran_ext`: Write the Fortran extension you are using in your project. The development of the **SlaD-Tool** has been focused on GFortran 4.6 and 8.1 (also 64 bits). It is recommended to compile your EMT model with one of these two extensions.
- `out_filename`: Write the name of the .out file that will be generated during the identification process. This name must be unique within the project. This is not going to save the data from the process, for that, **SlaD-Tool** will provide a final .txt with the admittance data and other results.
- `results_folder`: Write the name of the folder where the results will be saved. This is arbitrary and will be placed inside of the working directory.
- `Tinit`: This parameter defines the time at which the system reaches a steady state. It is set arbitrarily by the user based on prior observation of the system's transient behavior. In other words, it should correspond to the moment when the initial transient has passed, and all controllers and system dynamics are active. Figure 4 shows a practical example of how to identify this time in a simple transient. As illustrated, T_{init} would be the point where the system is stable and ready to receive perturbations for small-signal identification.

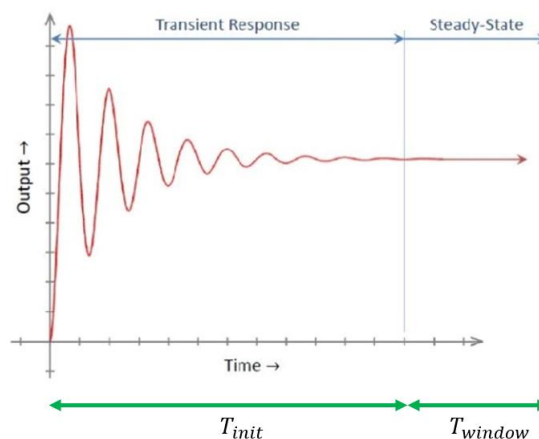


Figure 4. Details in the selection of T_{init} .

This also is the time where the snapshot (**.snp**) will be taken for the identification process. It is not necessary to disable or enable this function in your project since **SlaD-Tool** does it automatically.

- f_s : This parameter defines the desired frequency resolution in the frequency domain, i.e., the minimum frequency to be identified in the system. It directly determines the time window (T_{window}) used during the simulation, based on the inverse relationship:

$$f_s = \frac{1}{T_{window}}$$

Therefore, a lower resolution frequency f_s requires a longer time window T_{window} to accurately capture the system dynamics within that frequency range. When the f_s parameter is set, **the tool automatically** calculates the additional simulation time required to cover the necessary observation window. This total simulation time is referred to as T_{obs} , and is defined as:

$$T_{obs} = T_{init} + T_{window}$$

For example:

- If a 1 Hz spectral resolution is desired, the time window will be 1 second, so the simulation must run until $T_{init} + 1\text{ s}$.
- If a 0.1 Hz resolution is required, the window will be 10 seconds, and the total simulation time will be $T_{init} + 10\text{ s}$.

This parameter can also be interpreted as the **minimum frequency to be identified**, and its selection depends on the type of study being conducted, such as:

- Subsynchronous interactions (require lower frequencies, hence longer windows).
 - Harmonic interactions (may require higher frequencies, with shorter windows).
- f_{max} : Upper limit of the desired frequency band (the minimum is fixed by the previous parameter). Must be consistent with the type of study. It is crucial not to exceed the sampling limit to avoid aliasing errors:

$$f_{max} < \frac{f_{sampling}}{4} ; \text{ where } f_{sampling} = \frac{1}{step_{time}}$$

- f_{points} : This is the number of frequency points that the user wants to measure for the FD identification. In the experience of the authors, more than 150 points is recommended for accurate results.

When multi-tone perturbations are used, this number will specify the number of fundamental components that will be used for the identification. In this sense, $f_{\max}$ will not be longer required, and the maximum frequency will be fixed by the last component given by this parameter. For improved identification at low frequencies, a narrow spectrum is recommended (e.g., 1 to 6x50 Hz). However, this approach is not recommended for high-frequency analysis.

It is important to ensure consistency between the selected perturbation signal type and the corresponding configuration. If both settings are not aligned, the program will return an error.

Recommendation: The authors suggest using *single-tone* perturbations due to their higher accuracy.

- f_{scale} : This parameter selects between the two available options for the type of spaced measurements (just for single-tone strategy): 1 for logarithmically spaced measurements and 2 for linear measurements. In **SlaD-Tool**, the frequency vector for the desired frequency band is determined using the minimum and maximum frequency, along with this parameter. This vector can be modified to specify all desired frequencies for system perturbation in the *FDScanning.py* script, as it is shown in Figure 5.

```
# Frequency vector for scanning
fd0 = np.unique(
    np.round(np.logspace(np.log10(fs), np.log10(f_max), f_points))
)
```

Figure 5. Details in the frequency vector inside the module *FDScanning.py*.

- step_time : This is the discrete integration step defined by the user for EMT simulations and system identification. While arbitrary, it must be consistent with the type of study:
 - a. For harmonic interactions (high frequency), a high resolution is recommended, e.g., $<10 \mu\text{s}$.
 - b. For low-frequency interactions, a larger step can be used to speed up the study, with a maximum recommended value of $50 \mu\text{s}$.

3.2 Steady-State and Disturbance Source Data

This section includes variables required for the **ideal source** to properly identify the operating point of the element under study. In Figure 6 the available parameters to set the steady state data are shown. Here you must enter the general system parameters at the point of scan: f_0 , S_{base} and V_{base} .

```

ss_snap = 1           # Take a snapshot in 1st simulation: 1 yes and 0 not
Sbase = 100           # Base power (MVA)
Vbase = 230E3         # Base voltage (V)
f0 = 50              # Base frequency (Hz)
Vperturbation = 0.03  # Percentage of the nominal voltage for the perturbation

```

Figure 6. Second block of variables definition in `exe_SlaD.m`.

There is one available method to configure the steady-state operating point:

1. **Automatic calculation:** it is not necessary to define manually the operating point as the tool will automatically determine the steady-state. The variables `Vperturbation` and `Iperturbation` define the percentage of disturbance applied to voltage and current, respectively. These values should always remain between **0.01 and 0.03** to preserve linearity and ensure the system remains close to its nominal operating point.

NOTE: In both cases, it is mandatory to define `f0`, `Sbase` and `Vbase`.

Here the variable `ss_snap` allows to the user to select between taking a snapshot at `Tinit` (`ss_snap = 1`) or run the whole identification process from 0 to `Tobs` (`ss_snap = 0`). This is depending on the facilities of the project to take a snapshot or not. Consider that taking a snapshot (`ss_snap = 1`) accelerates the identification since the first transient and initialization are skipped.

Step 4: Select the Scanning Options

Next, you must select the available scanning options within the script. Figure 7 shows the steady-state and scanning methodologies integrated into the tool. All available combinations are not currently finished. **For the moment, just single-tone and voltage perturbation in all sequences are the available options.**

To properly tailor each methodology to your specific study, it is highly recommended to consult the reference article associated with the tool. Doing so will help you achieve more accurate and relevant results for your analysis.

```

# --> Available scanning strategies:
# 1 -> Voltage perturbation
# 2 -> Current perturbation (NOT AVAILABLE YET)
scanner_type = 1
# 1 -> Single-tone perturbation
# 2 -> PRBS perturbation (NOT AVAILABLE YET)
# 3 -> Multi-tone perturbation (NOT AVAILABLE YET)
signal_type = 1
# 1 -> ABC scan
# 2 -> dq0 scan
# 3 -> 0pn scan
scanner_selector = 2

```

Figure 7. Scanning options available in the **SlaD-Tool**.

Users can choose between different types of perturbation and identification configurations:

- Perturbation type:
 - **Voltage-based** (current-based not available yet).
 - Identification signal just in single-tone framework (either random binary or multi-frequency sine are not available yet).
- Reference sequence:
 - Identification can be performed in different **reference sequences** (e.g., positive-negative or dq), depending on the analysis requirements.

All available options are clearly numbered and described within the script, allowing for a very straightforward and user-friendly selection process.

Step 5: Finalize the Executable Script Configuration

The final part of the *exe_SlaD.py* script includes the necessary components to properly execute the integrated analysis modules (see Figure 8). Users must specify which analysis modules to activate in order to evaluate the stability and interaction characteristics of the identified matrices for both systems. The available modules include various techniques for small-signal analysis, passivity assessment, and frequency-domain stability evaluation.

The *FDScanning.py* automatically generates the variables *fd0*, *Ysys1* and *Ysys2*, so the user does not need to modify the variables within each function. Just comment or uncomment the desired functions. **DO NOT MODIFY THE IMPORTS SECTION AND THE "FDScanningTool" FUNCTION.**

NOTE: The ABC scanning strategy is not compatible with the stability modules.

```
# =====
# SIaD-Tool REPORT
# =====

gnc_result = Run_GNC(fd0, Ysys1, Ysys2,
                    outcomes_dir=results_folder,
                    scanner_selector=scanner_selector)

ma_results = Run_Modal_Analysis(fd0,
                                Ysys1,
                                Ysys2,
                                subsys_labels=("Sys1", "Sys2"),
                                outcomes_dir=results_folder)

PM_table = Run_Phase_Margins(fd0, Ysys1, Ysys2, scanner_selector)

PA_results = Run_Passivity_Analysis(fd0,
                                    Ysys1,
                                    Ysys2,
                                    outcomes_dir=results_folder)
```

Figure 8. Final setup for the *exe_SlaD.m*.

Step 6: Integrate the **SlaD** Block in Simulink

To integrate the block that enables system identification within your PSCAD™ model, follow these steps:

1. Open the *SlaD_Tool.pscx* file, located in the **root folder** where the tool was downloaded (see **Figure 1** for a reference of the folder structure).
2. Inside this file, you will find the block named “**SlaD-Tool**”, colored in **red** for easy identification.
3. This block should be **copied and pasted special** into your project’s EMT model (.pswx) at the point in the system where you want to perform the analysis.

NOTE: Remember to use the paste option “**Paste Transfer**” when you paste the block in your main canvas space. Otherwise, the block will be in red and not recognized by the project.

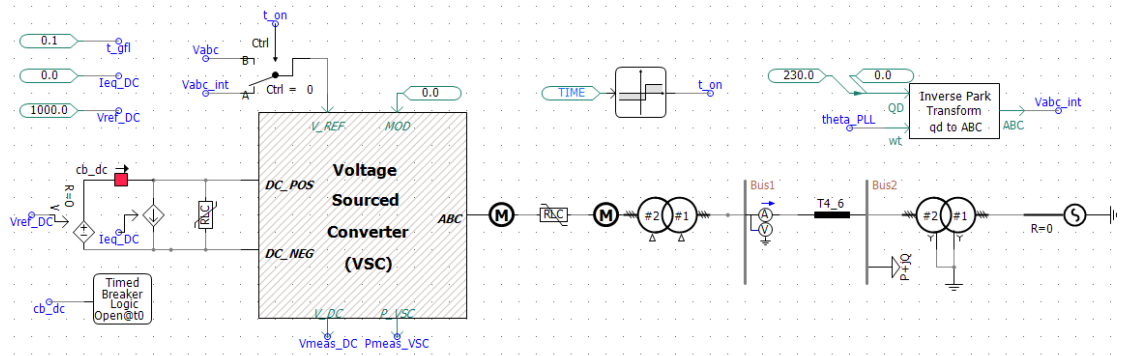
Figure 9 shows the visual location and appearance of the block within the original file.



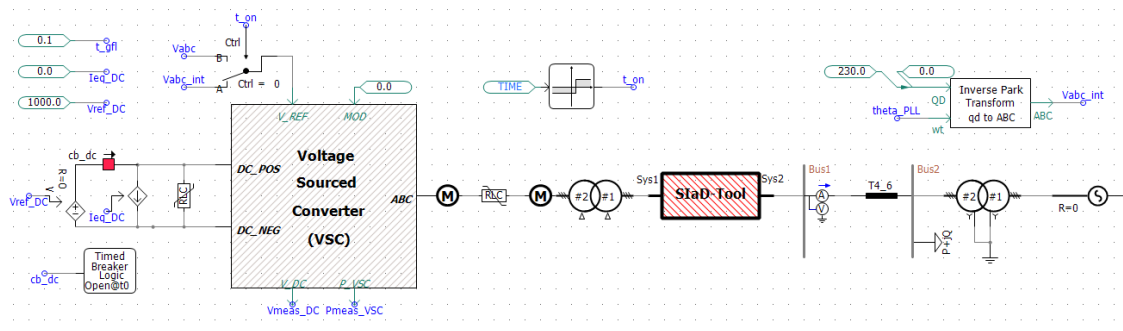
Figure 9. **SlaD**-Tool block.

Next, you must **properly connect the terminals** of the block to the **three-phase buses of system 1 (suggested for power electronics-based resources) and system 2 (suggested for grid or any other element)** where system identification is to be performed. This connection point should correspond to the location in the system where the frequency response is to be measured. Notice that the tool is made to determine the interactions between power electronics (left side) and the grid (right side), so it is recommended to follow the convention here proposed.

For example, in a typical case study of an IBR connected to the grid as is shown in Figure 10, the scanner is placed at the Point of Connection (PoC) between the Delta-Delta Transformer and the Bus 1. Then, in Figure 10-a), the initial EMT model is presented and next in Figure 10-b), the connection of the scanner is highlighted. Notice that the port Sys1 is connected to the power electronics resource and port Sys2 is connected to the rest of the system.



a) Scheme before the SlaD-Tool connection.



b) Scheme after the SlaD-Tool connection.

Figure 10. Scheme of connection for an IBR.

Step 7: Build and save the project

Once the block has been properly inserted in the .pswx file, you must build the full project again to make sure that all the functionalities of the tool have been compiled in the workspace. You must select the Build All option (see Figure 11).

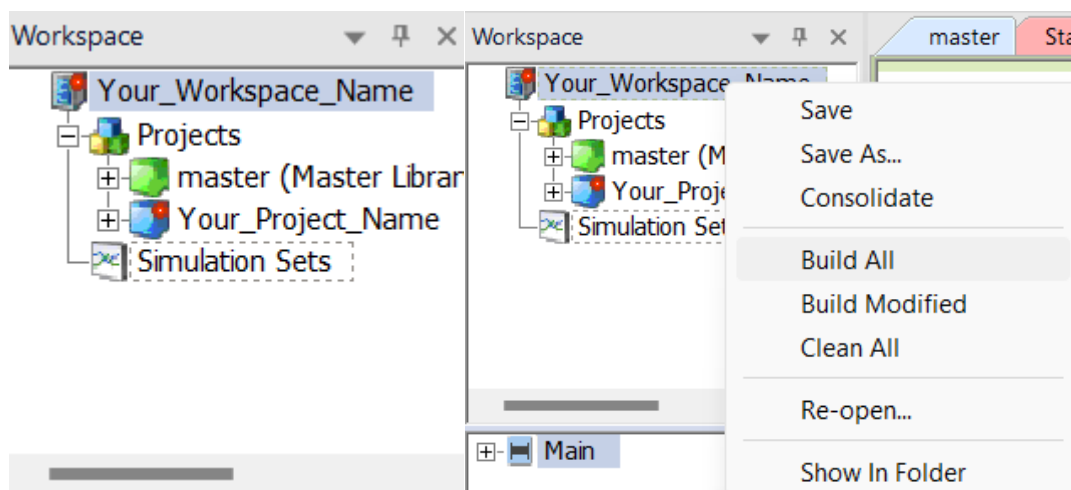


Figure 11. Build All option in PSCAD™.

Step 8: Running the Study and Viewing Results

Once all **7 steps** of this manual have been completed and you have already saved the full project, run the executable script (`exe_SlaD.py`). The tool will automatically perform the identification study and generate the real-time status of the analysis and will print the results in the terminal save the results in the desired folder, depending of the modules previously selected for the entire study.

The **identified matrices** will be saved in your selected folder as .txt files, named as Ysys1.txt and Ysys2.txt.

APPENDIX A: Important generalities within the process of SlaD-Tool.

In general, the tool has been tested in different projects but many bugs and implementation details could arise. In this Appendix, we will update the common errors or problems providing the founded solution for them.

CITCEA-UPC thanks you for downloading the SlaD Tool. We appreciate your interest and welcome any questions, suggestions, or collaboration proposals related to the use and development of the tool.

Who to Contact for Support or Follow-Up?

For questions, follow-up, or technical support regarding the **SlaD** tool, please contact:

 luis.reyes@upc.edu

Important Note

The SlaD tool is still under development. Any improvements, suggestions, or issues you identify are highly appreciated and should be reported to the authors to help **refine and enhance** the tool for the benefit of the community.

Acknowledgements

This work has received funding from the ADOrED project under the European Union's Horizon Europe Research and Innovation Programme under the Marie Skłodowska-Curie Grant Agreement No. 101073554.